

An introduction to ReQL

ReQL (RethinkDB Query Language) is regarded as a powerful query language for performing all sorts of operations on JSON documents. Compared to other NoSQL query languages, ReQL provides powerful features and is built on the following three key principles.

- **Embeds into the programming language:** All the queries are developed by using function calls in programming languages. There is absolutely no requirement for concatenation of strings or construction of specialised JSON objects for the database query.
- **All queries are chainable:** A user can start with a table and increment chain transformers to the query end using the '.' operator.
- **Queries are executed on the server:** Queries are made by the client on the respective machine, but once the user presses the *Run* command, the queries pass through the client machine to the server through the active network connection, and entire queries are executed on the database.

Features of ReQL

- **Efficient:** Like other database systems, ReQL supports all sorts of primary and secondary indexes for efficient access to data. Users can create compound indexes on the basis of arbitrary ReQL expressions in order to speed up the query process.
- **Parallel query execution:** Every user query is split across different cores of the CPU, server clusters or multiple data centres. RethinkDB and ReQL break the queries into stages and execute every stage in parallel; after completing the query, all data sets are combined to give the end result to the user.
- **Query optimisation:** ReQL is powerful enough as a query optimiser to maintain chains and alternative implementation plans to improve the overall performance of the database.
- **Other features:** ReQL builds

queries of arbitrary complexity; there is no new type of syntax or command for executing complex queries, and it allows modular programming when creating sub-queries and for handling user responses.

Data types

The ReQL data types can be categorised as follows:

- Basic data types
- RethinkDB specific data types
- Abstract data types
- Geometry data types

Basic data types

- *Number:* Any real number like 89, 4.561, -76, etc
- *Strings:* Any valid UTF-8 string like 'alphabet'
- *Booleans:* True/false
- *Objects:* JSON data objects
- *Arrays:* Elements of similar data types like [1 2 3], [red orange mango]

Specific data types

- *Databases:* RethinkDB databases
- *Tables:* RethinkDB database tables
- *Streams:* These list data types like arrays
- *Selections:* Subsets of tables
- *Pseudo types:* Different kinds of ReQL-specific data types which are generally composite, or special cases of other types like binary objects, times, geometry data types, grouped data

Abstract data types

- *Datum:* Used for most non-stream data types including basic data types, pseudo types, objects, and non-stream selections
- *Sequences:* These list data types like arrays, streams, selections and tables
- *Functions:* These are passed as parameters to various ReQL commands

Geometry data types

- *Points*
- *Lines*
- *Polygons*

Table 1: The differences between RethinkDB and PostgreSQL

Differences	RethinkDB	PostgreSQL
Licence	Apache License 2	PostgreSQL License
Supported operating systems	Linux, Windows, OS X	Android, Linux, OS X, Windows, FreeBSD, iOS
Programming language-base	C++, Python	C
Definition	Open source, scalable, distributed database to store JSON documents	Object-relational database management system
Cloud platform supported	Amazon AWS	Amazon EC2, Heroku, OpenShift, Digital Ocean, Windows Azure, Amazon RDS, Google App Engine
Replication mode	Asynchronous and synchronous replication	Master-slave, multi-master, asynchronous and synchronous replication
Locking model	MVCC	MVCC
Integrity model	MVCC	ACID, MVCC
Database model	Document-oriented, schemaless, NoSQL	Object-relational, JSON database, key-value
Query language	ReQL	SQL
Data types	JSON	BLOB
Partitioning methods	Declarative partitioning	Sharding